Efficient Truss Decomposition over Large Directed Graphs

Wensheng Luo
Hunan University
luowensheng@hnu.edu.cn

Qiaoyuan Yang
Peking University
2401111962@stu.pku.edu.cn

Yingli Zhou
The Chinese University of Hong Kong, Shenzhen
yinglizhou@link.cuhk.edu.cn

Fangyuan Zhang
Huawei Hong Kong Research Center
zhang.fangyuan@huawei.com

Yuan Xie
The University of Tokyo
xieyuan@g.ecc.u-tokyo.ac.jp

Yuanyuan Zeng
The Chinese University of Hong Kong, Shenzhen
zengyuanyuan@cuhk.edu.cn

Min Shi
Hunan University
shimin22@hnu.edu.cn

ABSTRACT

Dense subgraph discovery in directed graphs is a fundamental problem with many applications in network analysis. The D-truss model captures cohesive structures by enforcing both cycle-based and flow-based triangle constraints, but existing methods are computationally expensive because they repeatedly process the entire graph under different flow settings. In this paper, we propose PACT, a progressive framework for efficient D-truss decomposition. PACT organizes computation along increasing flow thresholds and reuses intermediate results to avoid redundant recomputation. To enable this process, we introduce anchored cycle trussness, which summarizes the cycle-based strength of an edge under a given flow constraint and evolves monotonically as the flow threshold increases. Building on PACT, we further incorporate an order-aware view and develop PACO, which exploits the induced edge order to compactly represent D-truss structures. We show that this order changes only locally across consecutive flow levels, which theoretically bounds and practically reduces the set of edges requiring refinement, further improving efficiency. Extensive experiments on large real-world directed graphs demonstrate that PACO significantly outperforms existing methods, achieving speedups of up to two orders of magnitude.

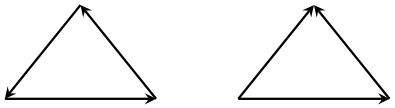
PVLDB Reference Format:
Wensheng Luo, Qiaoyuan Yang, Yingli Zhou, Fangyuan Zhang, Yuan Xie, Yuanyuan Zeng, and Min Shi. Efficient Truss Decomposition over Large Directed Graphs. PVLDB, 17(10): 2654 - 2667, 2024.
doi:10.14778/3675034.3675054

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/GearlessL/PACO.

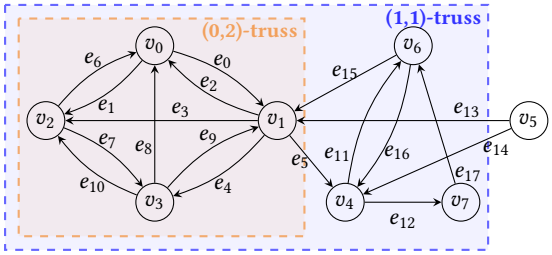
1 INTRODUCTION

In many real-world applications, directed graphs are used to model relationships among entities, such as social networks, e-commerce platforms, and financial systems, where edge directions encode essential semantic information. A fundamental problem in analyzing such graphs is the identification of dense subgraphs, commonly referred to as communities [15, 25], which serve as key primitives for understanding the underlying structural organization.

Recently, community search in directed graphs has attracted growing interest, aiming to identify dense subgraphs containing a given vertex or vertex set. To support this task, models such as D-core [8, 30, 33] and D-truss [29, 31, 49] have been proposed, among which truss-based models are particularly appealing for capturing higher-order cohesion via triangle-based edge support. In directed graphs, triangles are heterogeneous and can be classified into cycle and flow triangles [48], representing reciprocal and hierarchical interaction patterns, respectively, as shown in Fig. 1(a). Accordingly, a D-truss is defined as a maximal subgraph in which each edge participates in at least k_c cycle triangles and k_f flow triangles, enabling a finer-grained characterization of cohesive structures. Fig. 1(b) provides examples of a $(1,1)$-truss and a $(0,2)$-truss.



(a) Cycle triangle (left) and flow triangle (right)



(b) An example of a directed graph and its D-truss

Figure 1: Illustrations of directed triangles and D-truss.

D-truss has demonstrated practical value across a range of directed-network applications [24, 29, 31, 49]. In social networks, it facilitates the identification of tightly connected groups with mutual interactions as well as influence-oriented structures characterized by asymmetric following patterns. In associative word networks used in computational linguistics, D-truss reveals semantically cohesive neighborhoods around a query term, distinguishing symmetric associations from hierarchical reminder relationships [49]. More

Proceedings of the VLDB Endowment, Vol. 17, No. 10 ISSN 2150-8097.
doi:10.14778/3675034.3675054

broadly, D-truss is applicable to directed web, citation, and transaction networks, where capturing different forms of directional cohesion is essential for effective dense subgraph analysis [48].

The process of computing all D-trusses in a directed graph is referred to as D-truss decomposition, whose goal is to enumerate all non-empty (k_c, k_f)-trusses [31]. Most existing methods approach this task by treating each parameter pair (k_c, k_f) independently. In practice, they first determine the maximum feasible flow threshold $k_{f_{\max}}$, and then, for each k_f in this range, iteratively remove edges that participate in the fewest cycle triangles in the remaining graph until no edges remain. This iterative edge-removal procedure is commonly known as peeling. While straightforward, this strategy leads to substantial inefficiency. Processing different flow thresholds independently causes extensive redundant computation across successive k_f values, and each step repeatedly invokes expensive global peeling operations. This inefficiency is particularly problematic given a fundamental structural property of D-trusses: for a fixed k_c, the D-truss obtained under a larger k_f is always a subgraph of that under a smaller k_f. As a consequence, D-trusses at adjacent flow thresholds share a large portion of their structure, yet existing methods fail to exploit this overlap and instead recompute each decomposition from scratch.

This observation motivates a closer look at how the D-truss structure evolves as the flow threshold increases. Increasing k_f does not arbitrarily reshape the graph. Instead, the flow constraint monotonically removes infeasible edges while largely preserving the remaining structure. Under cycle-based constraints, the structural strength of surviving edges typically degrades gradually rather than collapsing abruptly. These properties indicate that the D-truss at a given flow threshold provides a strong basis for computing the result at the next threshold, enabling a progressive refinement strategy along the flow dimension. From a structural viewpoint, the peeling process at a fixed flow threshold implicitly orders edges by their robustness under cycle constraints: weakly supported edges are removed earlier, while stronger ones persist. As the flow threshold increases, this ordering remains largely intact and evolves in a localized manner, primarily affecting edges directly constrained by the tighter flow requirement. Consequently, D-truss decomposition can be viewed as maintaining an evolving edge order across flow thresholds, rather than repeatedly performing global refinement from scratch.

Our approaches. Guided by this insight, we propose a progressive framework for D-truss decomposition, termed PACT. Instead of treating each flow threshold as an independent task, PACT explicitly organizes computation along the flow dimension and views increasing flow thresholds as a sequence of closely related refinement steps. The framework builds upon both the nested structure of D-trusses and the observed stability of cycle-based structural strength across consecutive flow levels.

A central abstraction in PACT is anchored cycle trussness, which quantifies the structural robustness of an edge under a fixed flow constraint. For a given flow threshold, the anchored cycle trussness of an edge is defined as the highest cycle-based truss level at which the edge can be retained while respecting the flow constraint. This measure compactly summarizes cycle-based support within the feasible subgraph and, importantly, is monotonically non-increasing as the flow threshold increases. As such, anchored cycle trussness provides a unifying basis for organizing and updating D-truss structure across flow levels.

Building on this abstraction, PACT conducts two global peeling-based decompositions. One is performed at $k_f = 0$ to compute the anchored cycle trussness of all edges, and the other at $k_c = 0$ to obtain their maximum flow trussness. These results jointly define the initial feasible space and enable an efficient partitioning of edges by flow levels. In the *progressive phase*, PACT incrementally increases the flow threshold from k_f to $k_f + 1$. Edges that violate the updated flow constraint are efficiently identified via the precomputed flow classes and removed, yielding a strictly smaller candidate subgraph. The anchored cycle trussness values of the remaining edges are then refined to reestablish D-truss feasibility, driven solely by the local structural changes induced by these removals. By inheriting anchored cycle trussness values as tight upper bounds and operating on progressively shrinking subgraphs, PACT avoids redundant traversal of unaffected regions. As the flow threshold increases, both the candidate subgraph and the refinement scope contract monotonically, leading to decreasing update costs and enabling PACT to scale efficiently to large graphs.

Beyond progressive refinement, we introduce an order-aware view of D-truss computation and develop PACO, an order-aware algorithm built upon PACT. Under a fixed flow constraint, anchored cycle trussness induces a total order over edges that compactly represents the D-truss structure. As the flow threshold increases, D-truss evolution can be viewed as maintaining this order while only a limited number of edges are removed or locally weakened. To formalize this process, we leverage a framework for incremental graph analysis [12] to characterize how the edge order changes between consecutive flow thresholds. We show that this evolution is bounded: only edges involved in localized structural changes may alter their relative order, whereas most edges retain both their trussness values and order positions. This boundedness result provides the theoretical foundation for PACO, enabling refinement to be confined to a small set of affected edges and thereby improving efficiency on large directed graphs.

Contributions. We make the following contributions.

- We formulate progressive D-truss decomposition across increasing flow thresholds, revealing structural dependencies that are overlooked by existing approaches.
- We introduce an order-based structural perspective for understanding the evolution of D-trusses under varying flow constraints, and provide a theoretical analysis showing that this order evolution is bounded under incremental flow updates, establishing a formal foundation for efficient maintenance.
- We develop an order-aware refinement algorithm with strong efficiency guarantees, which avoids redundant global refinement by confining updates to a small affected region of the graph.
- We conduct extensive experimental evaluations on a diverse set of large real-world directed graphs, demonstrating that our method outperforms existing baselines. Specifically, our proposed method is up to two orders of magnitude faster than existing approaches.

2 RELATED WORK

In this section, we mainly review the existing works of decomposition for trusses and other cohesive subgraphs in large graphs.

2.1 Truss decomposition and maintenance

The k-truss is a widely used cohesive subgraph model for undirected graphs, defined as the maximal subgraph where every edge is part of at least $k-2$ triangles [9]. Truss decomposition identifies all k-trusses for all possible k values, with state-of-the-art serial in-memory and external-memory algorithms achieving a time complexity of $O(m^{1.5})$, where m is the number of edges [51]. Efficient parallel algorithms are discussed [22, 42, 46], and distributed algorithms leveraging MapReduce and BSP models are introduced [7]. Recently, Li et al. [27] proposed a tensor-based GPU framework for k-truss decomposition by leveraging directed graph representations and tensor operators for efficient support maintenance. A k-truss variant considering triangle connectivity models cohesive communities and can be computed by traversing the k-truss [20, 41, 53]. For truss maintenance, structures focusing on ordinary connectivity, triangle connectivity, and edge decomposition order have been proposed [20, 53, 56].

In directed graphs, triangles are classified into cycle and flow types [48], leading to the introduction of the D-truss model [31]. Liu et al. proposed a D-truss decomposition algorithm with time complexity $O(\min\{k_c, k_f\} \cdot m^{1.5})$ [31]. Tian et al. [50] study distributed D-truss decomposition for large directed graphs and propose several distributed peeling-based optimization techniques to improve scalability. Recently, an indexing method for dynamic directed graphs supports fully dynamic D-truss queries [49]. Liao et al. explored D-truss community search in streaming directed graphs [29], while Ai et al. examined community search considering triangle connectivity [1]. However, existing D-truss decomposition methods still face substantial performance challenges on large-scale graphs.

We note that our work shares certain high-level similarities with the dynamic D-truss maintenance framework in [49], such as order-aware processing strategies. However, the two works address fundamentally different problems. Different from [49], which focuses on dynamic D-truss maintenance after graph updates, our work studies progressive D-truss decomposition across increasing flow thresholds. The dynamic framework in [49] assumes that the D-truss decomposition has already been constructed and maintains truss structures under edge insertions and deletions. In contrast, our work addresses the decomposition problem itself, where the main challenge is to avoid redundant recomputation across consecutive k_f levels during iterative peeling. To this end, we develop a progressive refinement framework together with a dual-support propagation mechanism that jointly maintains cycle and flow constraints during decomposition. Moreover, unlike [50], which studies distributed computation, our work focuses on efficient single-machine algorithmic optimization.

Furthermore, truss models and their decompositions on bipartite graphs [52], uncertain graphs [47], and heterogeneous information networks [55] have also been extensively researched.

2.2 Other cohesive subgraphs

In addition to the k-truss, several other typical cohesive subgraphs in undirected graphs exist, including the k-core [5, 32], k-clique [11, 28], nucleus [43, 44], k-edge connected components [6, 19], and densest subgraph [18, 35, 57]. For example, the k-core [3] requires that each vertex in the subgraph has at least k neighbors. The k-clique [21, 28, 39] demands that k vertices in the subgraph are fully connected, while the densest subgraph [16, 18] is defined as the subgraph with the largest density, where density is the ratio of the number of edges to the number of vertices.

These models have also been extended to directed graphs. For instance, D-core [33] is the directed version of the k-core, considering in-degree and out-degree, and the directed densest subgraph [35, 38] extends undirected density to directed density. Moreover, cohesive subgraph models have been studied in other types of graphs, such as bi-core [36], biclique [37, 54], and quasi-biclique [34] in bipartite graphs, as well as (k, η)-core [4] and (k, τ)-clique [10] in uncertain graphs. However, due to the differences in these models, their methods cannot be directly applied to D-truss.

3 PROBLEM STATEMENT

Let $G = (V, E)$ be a directed, unweighted simple graph, where V and E denote the vertex and edge sets, respectively. Each edge $e = (u, v) \in E$ is directed from u to v; accordingly, u is an in-neighbor of v, and v is an out-neighbor of u. For any vertex $v \in V$, let $N^-(v)$ and $N^+(v)$ denote its sets of out-neighbors and in-neighbors, respectively. The out-degree and in-degree of v are defined as $d^-(v) = |N^-(v)|$ and $d^+(v) = |N^+(v)|$, and the degree of v is $d(v) = d^-(v) + d^+(v)$. We further denote by δ the maximum degree in G. Table 1 summarizes the notations frequently used throughout this paper.

Table 1: Notations and meanings.

Notation	Meaning
$G = (V, E)$	A directed graph G with vertex set V and edge set E
n, m	The numbers of vertices and edges in G resp.
$N^-(v), N^+(v)$	The out-neighbor set and in-neighbor set of a vertex $v \in G$ resp.
$d(v)$	The degree of a vertex $v \in G$
δ	The maximum degree of all vertices in G
$d^-(v), d^+(v)$	The out-degree and in-degree of a vertex $v \in G$ resp.
$csup(e, G), fsup(e, G)$	The cycle support and flow support of $e \in G$.
$\triangle_c(e), \triangle_f(e)$	the sets of cycle and flow neighbors of $e \in G$.

Before presenting the formal problem statement, we first introduce several key concepts. We consider a directed triangle consisting of three vertices. A *cycle triangle* is a triangle in which each vertex has exactly one incoming edge and one outgoing edge; it is denoted by $\triangle_c$. A *flow triangle* is a triangle in which the vertices have out-degrees 0, 1, and 2, and correspondingly in-degrees 2, 1, and 0; it is denoted by $\triangle_f$. Figure 1(a) illustrates an example of directed triangles, with the left side showing a cycle triangle and the right side displaying a flow triangle. For an edge (e), let $\triangle_c(e)$ and $\triangle_f(e)$ denote the sets of edges that form cycle triangles and flow triangles with e, respectively; the two accompanying edges in each triangle are called the cycle neighbors and flow neighbors of e. Building on these notions, we next define two types of support underlying D-truss.

Definition 1 (Cycle Support, Flow Support [31]). *Given a graph G and an edge e, the cycle support of e is the number of cycle triangles containing e in G, while the flow support of e in G is the number of flow triangles in G that contain e.*

We denote the cycle support and flow support of an edge e in G as $csup(e, G)$ and $fsup(e, G)$, respectively.

Definition 2 (D-truss [31]). *Given a directed graph $G = (V, E)$ and two integers k_c and k_f, a D-truss of G, also known as a (k_c, k_f)-truss, is a maximal subgraph $H = (V_H, E_H) \subseteq G$ such that the cycle support and flow support of edges in H are not less than k_c and k_f, respectively, i.e., $\forall e \in E_H, csup(e, H) \geq k_c$ and $fsup(e, H) \geq k_f$.*

The pair (k_c, k_f) is called the *trussness* of a (k_c, k_f)-truss and of its edges, where k_c and k_f are referred to as the *cycle trussness* and *flow trussness*, respectively. Since an edge may belong to multiple D-trusses, it can be associated with multiple trussness values. We denote the set of all trussness values of an edge e by $\mathcal{T}(e)$. A (k_c, k_f)-truss has several notable properties. First, for any given trussness pair (k_c, k_f), there exists at most one (k_c, k_f)-truss in G. Second, D-trusses exhibit a partial nesting property: if $k'_c \leq k_c$ and $k'_f \leq k_f$, then the (k'_c, k'_f)-truss is a subgraph of the (k_c, k_f)-truss. Based on these definitions and properties, we formally define the problem of *D-truss decomposition* as follows.

Problem 1 (D-truss Decomposition). *Given a directed graph G, the D-truss decomposition problem is to identify all D-trusses of G, that is, to compute the (k_c, k_f)-truss for every feasible pair (k_c, k_f).*

Example 1. *Figure 1(b) depicts a directed graph G with 18 edges, and Table 2 summarizes all D-trusses of G. The maximum cycle and flow thresholds are $k_{c_{\max}} = 1$ and $k_{f_{\max}} = 2$, respectively, yielding a total of five non-empty D-trusses for different combinations of (k_c, k_f). Each cell in the table lists the edge set induced by the corresponding D-truss. For instance, the subgraph induced by $\{e_1$-$e_4,\ e_6$-$e_{10}\}$ forms a $(0, 2)$-truss of G, while the subgraph induced by $\{e_0, e_1, e_3$-$e_9, e_{11}, e_{12}, e_{15}$-$e_{17}\}$ constitutes a $(1, 1)$-truss. The containment relationships among D-trusses are also evident: $(0, 2)$-truss $\subseteq (0, 1)$-truss $\subseteq (0, 0)$-truss, and similarly $(1, 1)$-truss $\subseteq (1, 0)$-truss. The goal of D-truss decomposition is to enumerate all such D-trusses of G, covering every valid pair of cycle and flow thresholds (k_c, k_f).*

Table 2: All non-empty D-truss of the directed graph shown in Figure 1(b).

k_c \ k_f	0	1	2
0	e_0-e_{17}	e_0-e_{17}	e_0, e_1, e_2, e_3, e_4 $e_6, e_7, e_8, e_9, e_{10}$
1	e_0, e_1, e_3-e_9, e_{11} $e_{12}, e_{15}, e_{16}, e_{17}$	e_0, e_1, e_3-e_9, e_{11} $e_{12}, e_{15}, e_{16}, e_{17}$	$\emptyset$

4 PROGRESSIVE D-TRUSS DECOMPOSITION

In this section, we first review the existing D-truss decomposition method and analyze its computational bottlenecks. To address these limitations, we propose a novel progressive computation framework, followed by an efficient refinement algorithm that leverages the nested structure of D-trusses.

Algorithm 1: Peeling-based D-truss Decomposition [31]

Input: a digraph $G = (V_G, E_G)$
Output: trussness for every edge in G

1 Compute $csup(e, G)$ and $fsup(e, G)$ for each edge in G;
2 $k_c \leftarrow 0$; $k_f \leftarrow 0$; $D(k_c, 0) \leftarrow G$, $L_e \leftarrow \emptyset$;
3 **while** $G \neq \emptyset$ **do** // global cycle peeling
4 $k_c \leftarrow k_c + 1$;
5 **foreach** *edge* $e = \{u, v\} \in E_G$ **do**
6 **if** $csup(e, G) < k_c$ **then** $L_e.push(e)$;
7 **while** $L_e \neq \emptyset$ **do**
8 $e \leftarrow L_e.pop()$ and delete it from G;
9 **foreach** *edge* e' *incident to* e *in* G **do**
10 Update $csup(e', G)$ and $fsup(e', G)$;
11 **if** $csup(e', G) < k_c$ **then** $L_e.push(e')$;
12 $D(k_c, 0) \leftarrow G$;
13 Let the maximum cycle truss as $k_{\max} \leftarrow k_c$;
14 **for** $k_c \leftarrow 0$ **to** $k_{\max}$ **do** // per-level flow peeling
15 Let the graph $H \leftarrow D(k_c, 0)$,$L_e \leftarrow \emptyset$;
16 Compute $fsup(e, H)$ for each edge in H;
17 **while** $H \neq \emptyset$ **do**
18 $e \leftarrow argmin_{e \in E_H}\{fsup(e, H)\}$, $k_f \leftarrow fsup(e, H)$;
19 $\tau(e) \leftarrow \tau(e) \cup (k_c, k_f)$, $L_e.push(e)$;
20 **while** $L_e \neq \emptyset$ **do**
21 $e \leftarrow L_e.pop()$ and remove e from H;
22 **foreach** *edge* e' *incident to* e *in* H **do**
23 Update $csup(e', H)$ and $fsup(e', H)$;
24 **if** $csup(e', H) < k_c$ *or* $csup(e', H) < k_f$ **then**
25 $T(e') \leftarrow T(e') \cup \{(k_c, k_f)\}$, $L_e.push(e')$;
26 **return** $\{\tau(e) | e \in E_G\}$;

4.1 Existing algorithm and its limitations

The state-of-the-art approach, proposed by Liu et al. [31], treats D-truss decomposition as a two-dimensional peeling problem. The trussness of an edge e is represented as a set of pairs $\tau(e) = \{(k_c, k_f)\}$. The method outlined in Algorithm 1 proceeds in two phases: (1) *Global Cycle Peeling:* It first determines the maximum cycle trussness $k_{c_{\max}}$ by iteratively removing edges with insufficient cycle support (*csup*) (lines 3–12); (2) *Per-Level Flow Peeling:* For each $k_c \in [0, k_{c_{\max}}]$, it computes the maximum flow trussness k_f for every edge in the subgraph $D(k_c, 0)$ through a secondary peeling process based on flow support ($fsup$), where $D(k_c, 0)$ is the induced subgraph consisting of all edges with cycle trussness at least k_c (lines 14-25). Once all k_c values have been processed, the trussnesses of all edges are obtained (line 26).

Limitations: Algorithm 1 begins by determining the range of possible k_c values in the entire graph. For each k_c, it computes the k_f value of every edge. This process relies on truss decomposition in undirected graphs [51], which has a time complexity of $O(m^{1.5})$. Therefore, the overall complexity of Algorithm 1 is $O(k_{c_{max}} \cdot m^{1.5})$. However, the algorithm suffers from substantial redundant computation: for each k_c, it not only re-initializes the support values (i.e., $fsup$) of all edges but also traverses the entire graph to recompute

the new k_c, leading to significant practical overhead. Although this overhead is theoretically covered by the $O(m^{1.5})$ bound, the actual cost in practice can be considerable.

4.2 Progressive D-truss Computation

A key property of D-truss is its *nested structure*. For any two parameter pairs (k_c, k_f) and (k'_c, k'_f) with $k_c \le k'_c$ and $k_f \le k'_f$, the (k'_c, k'_f)-truss is a subgraph of the (k_c, k_f)-truss. This monotonic containment holds jointly with respect to both the cycle and flow thresholds. As a direct consequence, we focus on the following special case.

Observation 1. *Fix a cycle threshold k_c. As the flow threshold k_f increases, the associated D-trusses form a nested sequence:*

$$(k_c, \tau)\text{-truss} \subseteq (k_c, \tau-1)\text{-truss} \subseteq \cdots \subseteq (k_c, 0)\text{-truss},$$

where τ denotes the maximum flow threshold for which the (k_c, k_f)-truss is non-empty.

This monotonicity ensures that denser D-trusses are always embedded in sparser ones. Consequently, intermediate results obtained at lower flow thresholds can be reused at higher thresholds, enabling an incremental computation paradigm. In contrast, existing methods typically treat each (k_c, k_f) pair or each flow threshold as an independent instance and recompute from scratch.

4.2.1 Progressive Computation Strategy. Motivated by the nested nature of D-trusses, we propose a **progressive computation workflow** that processes flow thresholds in increasing order, $k_f = 0, 1, 2, \ldots$, and incrementally derives the truss structure at level $k_f + 1$ from the stabilized result at level k_f. At each stage, the $(*, k_f)$-truss is refined under a stricter flow constraint, thereby fully exploiting the containment relationship between consecutive levels.

However, tightening the flow threshold from k_f to $k_f + 1$ may invalidate a subset of edges that previously satisfied the D-truss conditions. The removal of these edges can, in turn, reduce the effective cycle support of adjacent edges, triggering a cascading propagation of updates. Accurately capturing and bounding this propagation effect, without repeatedly invoking global peeling procedures, constitutes the central technical challenge of progressive computation.

To address this challenge, we introduce a quantitative measure that characterizes the structural strength of each edge under a fixed flow threshold. This measure enables fine-grained tracking of local stability and provides a principled basis for bounding the scope of updates during refinement.

Definition 3 (Anchored Cycle Trussness). *Given a flow threshold k_f, the **anchored cycle trussness** of an edge e, denoted by $\tau_c(e, k_f)$, is defined as the maximum integer k_c such that e belongs to a non-empty (k_c, k_f)-truss.*

Under this definition, D-truss decomposition can be viewed as the problem of computing $\tau_c(e, k_f)$ for every edge e at each flow level k_f. Empirically, we observe that for the vast majority of edges, the anchored cycle trussness changes only marginally between consecutive flow thresholds k_f and $k_f + 1$, and that the resulting values exhibit substantial overlap across levels. This phenomenon

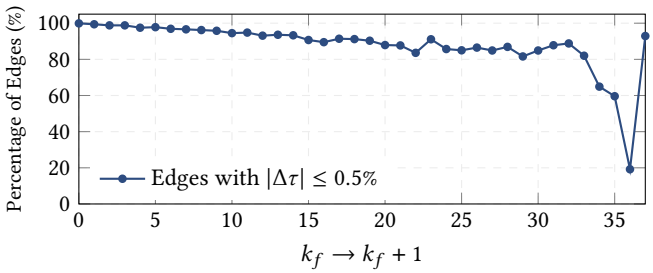


Figure 2: Continuity of anchored cycle trussness across consecutive flow thresholds on Email dataset.

is illustrated in Fig. 2 on the Email dataset [26]. Specifically, we measure the relative change $\Delta\tau = \frac{|\tau_c(e,k_f+1)-\tau_c(e,k_f)|}{\tau_c(e,k_f)}$, and report the fraction of edges whose relative variation is below 0.5%. For flow thresholds up to $k_f = 33$, more than 80% of edges satisfy this criterion, indicating strong incremental stability of anchored cycle trussness. These observations suggest that $\tau_c(e, k_f)$ provides a tight upper bound for $\tau_c(e, k_f + 1)$, and naturally justifies a refinement-based computation model.

4.2.2 Dual-support-based Refinement. For all edges, the transition from $\tau_c(\cdot, k_f)$ to $\tau_c(\cdot, k_f + 1)$ is realized by iteratively refining edge states until they reach a *structural equilibrium* consistent with the D-truss definition. We characterize this equilibrium through the following dual support condition.

Property 1 (Dual Support Condition). *Fix a flow threshold k_f. The anchored cycle trussness $\tau_c(e, k_f)$ of an edge e is the maximum integer t such that the following two conditions hold:*

1. ***Cycle support:** e is contained in at least t cycle triangles whose edges all have anchored cycle trussness at least t;*
2. ***Flow support:** e is contained in at least k_f flow triangles whose edges all have anchored cycle trussness at least t.*

Equivalently,

$$\tau_c(e, k_f) = \max\left\{ t \;\middle|\; \begin{array}{l} \left|\{\triangle \in \triangle_c(e) \mid \min_{e' \in \triangle} \tau_c(e', k_f) \ge t\}\right| \ge t, \\ \left|\{\triangle \in \triangle_f(e) \mid \min_{e' \in \triangle} \tau_c(e', k_f) \ge t\}\right| \ge k_f \end{array} \right\}.$$

Proof. An edge e belongs to a (t, k_f)-truss if and only if it is supported by at least t cycle triangles and at least k_f flow triangles whose edges remain in the truss. All edges in these supporting triangles therefore satisfy $\tau_c(e', k_f) \ge t$. Conversely, if e satisfies the above cycle and flow support conditions for some t, then during the peeling process enforcing the (t, k_f)-truss constraints, e cannot be removed since neither support condition is violated. Hence, e belongs to a non-empty (t, k_f)-truss and $\tau_c(e, k_f) \ge t$. Taking the maximum such t completes the proof. □

To operationalize Property 1, we define two local operators that propagate cycle and flow support.

Definition 4 (Cycle and Flow Operators). *Fix a flow threshold k_f. For each edge e, define*

$$C_e(k_f) = \max\left\{ t \mid \left|\{\triangle \in \triangle_c(e) : \min_{e' \in \triangle} \tau_c(e', k_f) \ge t\}\right| \ge t \right\},$$

$$\mathcal{F}_e(k_f) = \max\left\{ t \mid \left|\{\triangle \in \triangle_f(e) : \min_{e' \in \triangle} \tau_c(e', k_f) \ge t\}\right| \ge k_f \right\}.$$

Intuitively, $C_e(k_f)$ and $\mathcal{F}_e(k_f)$ quantify the maximal cycle and flow support that edge e can sustain under the current τ_c values. These operators replace explicit edge removal with numerical updates of trussness, transforming combinatorial peeling into an iterative refinement process.

Lemma 4.1. *Let $\tau_c^{(0)}(e,k_f)$ be any valid upper bound on $\tau_c(e,k_f)$, e.g., $\tau_c(e,k_f-1)$. For each iteration $r \geq 0$, update*

$$\tau_c^{(r+1)}(e,k_f) = \min\{C_e(k_f), \mathcal{F}_e(k_f)\}.$$

Then $\tau_c^{(r)}(e,k_f)$ converges in finitely many steps to $\tau_c(e,k_f)$.

Proof. Property 1 implies that $\tau_c(e,k_f)$ is a fixed point of the update rule. Moreover, both operators are monotone and the updates generate a non-increasing sequence bounded below by zero. Therefore, the sequence converges to the maximum fixed point, which is exactly $\tau_c(e,k_f)$. □

Lemma 4.1 underpins our progressive algorithm by guaranteeing that the anchored cycle trussness at flow level k_f serves as a sound and tight initialization for level k_f+1, enabling efficient refinement across increasing flow thresholds.

4.2.3 Incremental Refinement Algorithm. Building on the progressive computation framework, we design an incremental algorithm to derive the anchored cycle trussness of all edges at flow threshold k_f+1 directly from the results obtained at threshold k_f. The core idea is to reuse the results of $(*,k_f)$-truss, eliminate edges that cannot survive the stricter flow constraint, and then refine the inherited anchored cycle trussness values until all constraints at the new threshold are satisfied. In this way, the results computed at level k_f provide tight initial estimates and substantially reduce redundant computation.

The refinement from k_f to k_f+1 proceeds as follows. Given the anchored cycle trussness values at threshold k_f, we first remove all edges that violate the flow requirement at the higher threshold. Specifically, edges whose maximum flow trussness equals k_f cannot appear in any $(*,k_f+1)$-truss and are therefore eliminated. This produces an initial candidate subgraph for threshold k_f+1. Starting from this subgraph, we iteratively refine the anchored cycle trussness values of the remaining edges until both the cycle and flow constraints at threshold k_f+1 are satisfied.

To efficiently identify edges that must be removed when the flow threshold increases, we group edges according to their flow trussness when no cycle constraint is imposed. This leads to the notion of a *flow class*.

Definition 5 (Flow Class). *The k_f-flow class of a graph G, denoted by Φ_{k_f}, is the set of edges whose maximum flow trussness equals k_f when no cycle constraint is imposed, i.e., $\Phi_{k_f} = \{\, e \in E \mid \tau_f(e,0) = k_f \,\}$.*

Example 2. *Consider the directed graph in Fig. 1(b). When $k_c = 0$, edges e_0–e_4 and e_6–e_{10} have flow trussness 2, whereas all remaining edges have flow trussness 1. Accordingly, the graph is partitioned into two non-empty flow classes, $\Phi_2 = \{e_0$–e_4, e_6–$e_{10}\}$ and $\Phi_1 = \{e_5, e_{11}$–$e_{17}\}$, while Φ_0 is empty.*

In addition to removing edges via flow classes, we further observe that for some flow thresholds the refinement step can be skipped entirely. This occurs when increasing the flow threshold does not invalidate case(2) of Property 1 and the corresponding flow class is empty. In this case, the key factor is how many flow triangles continue to support an edge's anchored cycle trussness as the flow threshold increases. For convenience, we use $\Lambda_f(e,k_f)$ to denote the number of flow triangles containing e whose minimum anchored cycle trussness among all edges in the triangle is no smaller than that of e. That is, $\Lambda_f(e,k_f) = |\{\triangle \in \triangle_f(e) | \min_{e' \in \triangle} \tau_c(e',k_f) \geq \tau_c(e,k_f)\}|$.

Lemma 4.2 (k_f pruning criterion). *Let k_f be a flow threshold. Assume that the k_f-flow class is empty, i.e., $\Phi_{k_f} = \emptyset$. If for every edge e, $\Lambda_f(e,k_f) \geq k_f+1$, then increasing the flow threshold from k_f to k_f+1 does not change the anchored cycle trussness of any edge. That is, $\tau_c(e,k_f+1) = \tau_c(e,k_f)$ for all edges e.*

Proof. Fix an arbitrary edge e and let $t = \tau_c(e,k_f)$. By Property 1, t satisfies both the cycle and flow constraints for e at threshold k_f. Since $\Phi_{k_f} = \emptyset$, increasing the flow threshold to k_f+1 does not remove any edge, and thus all cycle and flow triangles containing e remain unchanged. By assumption, e is supported by at least k_f+1 flow triangles whose minimum anchored cycle trussness is at least t. Because anchored cycle trussness is non-increasing with respect to k_f, these supports remain valid at threshold k_f+1. Hence, t remains feasible for e at threshold k_f+1, implying $\tau_c(e,k_f+1) \geq t$. By monotonicity, $\tau_c(e,k_f+1) = t$. Since e is arbitrary, the claim holds for all edges. □

Algorithm 2: Incremental Refinement

Input: Graph $G=(V,E)$ and $\{\tau_c(e,k_f) | e \in G\}$
Output: $\tau_c(e,k_f+1)$ for all $e \in E$

```
1  Flag ← false, active ← ∅, H ← (0,k_f)-truss of G;
2  if Φ_{k_f} ≠ ∅ then Removing e ∈ Φ_{k_f} from H, Flag ← true ;
3  for e ∈ H do
4      τ_c(e,k_f+1) ← τ_c(e,k_f), active[e] ← true;
5      if Λ_f(e,k_f) < k_f+1 then Flag ← true;
6  while Flag do
7      Flag ← false;
8      for e ∈ H ∧ active[e] = true do
9          temp ← min{C_e(k_f+1), F_e(k_f+1)};
10         active[e] ← false;
11         if temp < τ_c(e,k_f+1) then
12             τ_c(e,k_f+1) ← temp, Flag ← true;
13             for edge e' incident to e in H do
14                 active[e'] ← true;
15 return {τ_c(e,k_f+1) | ∀e ∈ H} ;
```

Algorithm 2 presents the pseudocode for the incremental computation of anchored cycle trussness. The algorithm updates the anchored cycle trussness of all edges when the flow threshold increases from k_f to k_f+1. At the new threshold k_f+1, edges belonging to the k_f-flow class are first removed to construct the current subgraph H (line 2). For each remaining edge $e \in H$, its anchored cycle trussness $\tau_c(e,k_f+1)$ is initialized to the value inherited from the previous threshold, namely $\tau_c(e,k_f)$ (lines 3–4).

The boolean variable *active* records whether an edge may require further refinement, while the flag *Flag* indicates whether the algorithm has converged (line 1). If the pruning conditions in Lemma 4.2 are satisfied (lines 2 and 5), namely that the k_f-flow class is empty, and all edges meet the flow–cycle support requirement, the algorithm terminates early, as no anchored cycle trussness value will change when moving to threshold $k_f + 1$. Otherwise, the algorithm iteratively refines the anchored cycle trussness values. Whenever an edge in H is marked as potentially active, its cycle and flow operators are recomputed (lines 8–9). If the newly computed value differs from the current anchored cycle trussness, the value is updated and all adjacent edges in H are marked as *active* (lines 11–15), allowing updates to propagate locally. This refinement process repeats until no further changes occur, at which point the anchored cycle trussness values have converged for the current flow threshold.

Time complexity. The time complexity of Algorithm 2 can be formally stated as follows.

Theorem 4.3. *Given a directed graph $G = (V, E)$ and the anchored cycle trussness of all edges for k_f, let H denote the induced subgraph corresponding to the $(0, k_f + 1)$-truss of G. The time complexity of Algorithm 2 is $O(m_H \cdot \alpha_H^2)$, where m_H is the number od edegs in H and α_H is the degeneracy of H.*

Proof. Algorithm 2 incrementally refines anchored cycle trussness from flow threshold k_f to $k_f + 1$ on the induced subgraph H of the $(0, k_f + 1)$-truss. After removing invalid edges, the algorithm iteratively updates only affected edges in H. For each edge, recomputing the cycle and flow operators depends only on its incident triangles. Hence, one refinement round costs $O(T_H)$, where T_H is the number of triangles in H. Under a degeneracy ordering, each edge participates in at most $O(\alpha_H)$ triangles, implying $T_H = O(m_H \cdot \alpha_H)$. Therefore, each refinement round runs in $O(m_H \cdot \alpha_H)$ time. Moreover, an edge can lose support at most $O(\alpha_H)$ times, so the number of refinement rounds is also bounded by $O(\alpha_H)$. Combining the above bounds, the overall time complexity is $O(m_H \cdot \alpha_H^2)$. □

4.3 Overall Procedure

The overall algorithm, termed Progressively Anchored Cycle Trussness computation (PACT), computes the D-truss decomposition by combining a peeling-based initialization with an incremental refinement process.

To avoid costly refinement when the number of active edges is maximal, PACT initializes both thresholds at zero using peeling. Specifically, $\tau_c(e, 0)$ is computed via a standard k-truss peeling procedure based on cycle support, where edges violating the current threshold are iteratively removed and supports are updated accordingly. The use of bin-sort [3] enables efficient support maintenance. Similarly, $\tau_f(e, 0)$ is obtained by peeling under the minimal cycle constraint $k_c = 0$, yielding both the initial flow trussness values and the corresponding flow classes.

With $\tau_c(e, 0)$ and $\tau_f(e, 0)$ initialized, Algorithm 3 enters the refinement phase. The algorithm first identifies the maximum flow level $k_{f_{\max}}$ and the associated flow classes Φ_ℓ (line 1) and then integrates the precomputed initialization results (line 2). For each flow level $k_f > 0$, the refinement procedure in Algorithm 2 incrementally updates trussness values on progressively sparser subgraphs, reusing intermediate states and avoiding redundant computation (lines 3-4). The algorithm finally outputs $\tau_c(e, k_f)$ for all edges across all flow levels from 0 to $k_{f_{\max}}$.

Algorithm 3: PACT

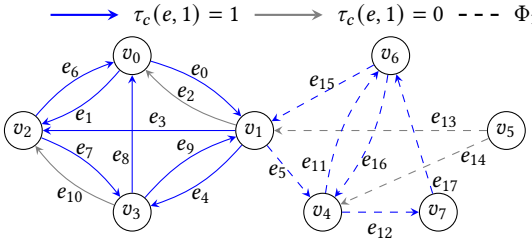
Input: $G = (V, E)$
Output: $\tau_c(e, k_f)$ for all $e \in E$, $k_f \geq 0$
// (1) truss decompisition with $k_c = 0$.
1 Compute $k_{f_{\max}}$ and $\{\Phi_\ell | \ell \in [0, k_{f_{max}}]\}$;
// (2) truss decompisition with $k_f = 0$.
2 Compute $\{\tau_c(e, 0)\}$;
// (3) anchored cycle trussness with $k_f > 0$.
3 **for** $k_f \leftarrow 1$ **to** $k_{f_{max}}$ **do**
4 Invoke Alg. 2 to obtain $\{\tau_c(e, k_f)\}$ from $\{\tau_c(e, k_f - 1)\}$;
5 **return** $\{\tau_c(e, k_f)\}$ for $k_f = 0, \cdots, k_{f_{max}}$;

Example 3. *Recall the directed graph in Fig. 1(b). We use this example to illustrate the whole process of Algorithm 3. As shown in Example 2, the non-empty flow classes are $\Phi_2 = \{e_0, \ldots, e_4, e_6, \ldots, e_{10}\}$ and $\Phi_1 = \{e_5, e_{11}, \ldots, e_{17}\}$, while $\Phi_0 = \emptyset$. At $k_f = 0$, the anchored cycle trussness values are $\tau_c(e, 0) = 0$ for $e_2, e_{10}, e_{13}, e_{14}$, and $\tau_c(e, 0) = 1$ for all remaining edges. Since $\Phi_0 = \emptyset$, increasing the flow threshold from $k_f = 0$ to $k_f = 1$ removes no edge, and thus $\tau_c(e, 1) = \tau_c(e, 0)$ for all edges. We next increase the flow threshold from $k_f = 1$ to $k_f = 2$. All edges in Φ_1 are first removed because they cannot belong to any $(*, 2)$-truss. The remaining candidate subgraph is induced by $\{e_0, e_1, e_2, e_3, e_4, e_6, e_7, e_8, e_9, e_{10}\}$, as shown in Fig. 3(b). For each remaining edge e, Algorithm 2 initializes $\tau_c^{(0)}(e, 2) = \tau_c(e, 1)$ using the previous flow level as an upper bound.*

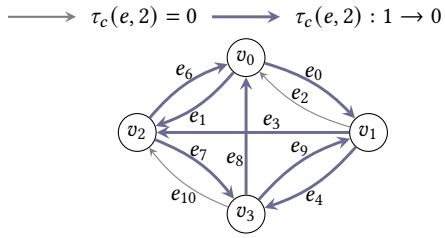
The refinement then repeatedly applies $\tau_c(e, 2) = \min\{C_e(2), \mathcal{F}_e(2)\}$ until convergence. After removing Φ_1, the dual-support condition in Property 1 is no longer satisfied at $k_f = 2$. Consequently, all remaining edges with $\tau_c(e, 2) = 1$ are refined to 0, while edges already equal to 0 remain unchanged. Since the update sequence is non-increasing and bounded below by 0, one additional scan detects no further change, and the refinement converges to the fixed point characterized by Property 1.

Time Complexity. The time complexity of Algorithm 3 consists of an initialization phase followed by a progressive refinement phase. In the initialization phase, we compute the initial anchored cycle trussness $\tau_c(e, 0)$, the maximum flow level $k_{f_{\max}}$, and all flow classes Φ_{k_f} via peeling-based truss decomposition. This phase runs in $O(m^{1.5})$ time in the worst case. After initialization, the algorithm iteratively processes flow levels $k_f = 1, \ldots, k_{f_{\max}}$. At each level, refinement is performed on the induced subgraph $H_{k_f} = H_{k_f - 1} \setminus \Phi_{k_f - 1}$. According to Theorem 4.3, the worst case time complexity at level k_f is $O(m_{H_{k_f}} \cdot \alpha_{H_{k_f}}^2)$, where $m_{H_{k_f}}$ and $\alpha_{H_{k_f}}$ denote the number of edges and the degeneracy of H_{k_f}, respectively. A naive aggregation over all flow levels yields $O(m^{1.5} + k_{f_{\max}} \cdot m \cdot \alpha^2)$, which simplifies to $O(k_{f_{\max}} \cdot m \cdot \alpha^2)$ in the worst case.

Note that the initialization stage still requires a global peeling procedure to construct the initial D-truss decomposition, which is fundamentally shared by existing peeling-based D-truss methods due to the need for global triangle-support computation and support maintenance. Therefore, our optimization mainly focuses on



(a) Anchored trussness with $k_f = 1$



(b) Removing edges in Φ_1 and refine τ_c with $k_f = 2$

Figure 3: Illustration of the refinement process from $k_f = 1$ to $k_f = 2$.

reducing redundant recomputation across different k_f stages rather than the one-time initialization cost itself. By incrementally reusing intermediate decomposition states, the proposed method avoids repeatedly executing independent peeling procedures from scratch and thus achieves substantial overall acceleration. However, the effectiveness of the progressive refinement strategy relies on the locality of structural changes. In worst-case scenarios with extremely dense graphs or large-scale propagation of support updates, the refinement process may approach a near-global peeling procedure. For example, when $m = \Theta(n^2)$ and $\alpha_H = \Theta(n)$, the refinement complexity becomes $O(m_H \alpha_H^2) = O(m^2)$. In addition, if the initial decomposition already coincides with the final result, the runtime is naturally dominated by the initialization stage itself.

5 ORDER-BASED D-TRUSS COMPUTATION

This section revisits the PACT algorithm from an order-based perspective. We show that progressive D-truss decomposition can be viewed as maintaining an edge ordering induced by anchored cycle trussness as the flow threshold increases. By analyzing how this order evolves across consecutive flow thresholds, we bound its changes and use this insight to reduce redundant refinement, improving the efficiency of D-truss computation. Replacing the original refinement procedure (Algorithm 2) in PACT with this order-aware strategy yields a more efficient algorithm, denoted as PACO.

5.1 Edge Ordering by Anchored Cycle Trussness

As discussed above, the execution of PACT can be viewed as incrementally maintaining an ordering of edges induced by anchored cycle trussness as the flow threshold increases. For a fixed flow threshold k_f, each edge e is assigned a scalar value $\tau_c(e, k_f)$, which quantifies its structural strength in the cycle dimension under the given flow constraint. These values naturally induce a total order over edges.

From this order-based viewpoint, D-truss decomposition is no longer interpreted as a sequence of independent refinement procedures, but rather as a process of order evolution. When the flow threshold increases from k_f to $k_f + 1$, edges that violate the stricter flow constraint are removed, and the anchored cycle trussness of the remaining edges may decrease. Both effects jointly induce a reordering of edges that captures localized structural changes in the graph.

While PACT realizes this evolution through progressive refinement, it treats each refinement step independently and does not explicitly exploit the relationship between the edge orders at consecutive flow thresholds. If such relationships can be characterized, redundant refinement can be further eliminated. Motivated by this observation, we formalize the notion of an edge order induced by anchored cycle trussness and use it as the basis for analyzing order evolution across flow thresholds.

Definition 6 (Anchored Cycle Order). *Fix a flow threshold k_f. The* anchored cycle order *at k_f, denoted by O_{k_f}, is a non-descending sequence of all edges in E, sorted by their anchored cycle trussness values $\tau_c(e, k_f)$. Formally, for any two edges e_i and e_j in O_{k_f}, e_i precedes e_j if $\tau_c(e_i, k_f) < \tau_c(e_j, k_f)$, or if $\tau_c(e_i, k_f) = \tau_c(e_j, k_f)$ and a fixed tie-breaking rule is applied.*

Table 3: The anchored cycle order for graph G.

Order	Edges ($\tau_c(k_f, e_i) = 0$, $\tau_c(k_f, e_i) = 1$)
$O_{k_f=0}$	$e_2, e_{10}, e_{13}, e_{14}$, $e_0, e_1, e_3, e_4, e_6, e_7, e_8, e_9, e_{11}, e_{12}, e_{15}, e_{16}, e_{17}$
$O_{k_f=1}$	$e_2, e_{10}, e_{13}, e_{14}$, $e_0, e_1, e_3, e_4, e_6, e_7, e_8, e_9, e_{11}, e_{12}, e_{15}, e_{16}, e_{17}$
$O_{k_f=2}$	$e_0, e_1, e_2, e_3, e_4, e_6, e_7, e_8, e_9, e_{10}$

Table 3 presents the anchored cycle order of the directed graph in Fig. 1(b), where edges with identical anchored cycle trussness values appear contiguously in O_{k_f} and are indicated by the same color. Consequently, the anchored cycle order provides a compact representation of the entire D-truss hierarchy at flow threshold k_f. Under this representation, a (k_c, k_f)-truss is exactly the prefix of O_{k_f} containing all edges with anchored cycle trussness at least k_c.

This order-based view enables a refined characterization of how D-truss structure evolves across flow thresholds. When the flow threshold increases from k_f to $k_f + 1$, edge removals and decreases in anchored cycle trussness jointly induce a transformation of O_{k_f} into O_{k_f+1}. Rather than recomputing trussness values from scratch, this perspective allows the evolution of D-trusses to be analyzed directly in terms of changes to the edge order. In the following subsection, we show that such order evolution is tightly bounded.

5.2 Boundedness of Order Evolution

Boundedness is a standard theoretical criterion for assessing the efficiency of incremental algorithms. It captures whether the cost of maintaining a query result after an update can be bounded by the size of the update itself and the induced change in the output.

We briefly introduce the boundedness framework and then analyze the evolution of anchored cycle order across consecutive flow thresholds.

Notations. Let G be a directed graph, and let $Q(G)$ denote the anchored cycle order at a given flow threshold. An update ΔG corresponds to increasing the flow threshold from k_f to $k_f + 1$, which removes all edges in the k_f-flow class. The updated graph is denoted by $G \oplus \Delta G$, and the corresponding change in the output order is denoted by ΔR, such that $Q(G \oplus \Delta G) = Q(G) \oplus \Delta R$.

Boundedness. Following the classical framework [40], the effectiveness of an incremental algorithm is evaluated using the metric CHANGED, defined as $\mathsf{CHANGED} = \Delta G \cup \Delta R$, with $|\mathsf{CHANGED}| = |\Delta G| + |\Delta R|$. This quantity represents a lower bound on the work required to maintain the query result.

Definition 7 (Boundedness [40, 56]). *An incremental algorithm is* bounded *if there exists a polynomial function $f(\cdot)$ and a constant k such that, for any graph G and update ΔG, the update cost is bounded by $O\big(f\big(|N_k(\mathsf{CHANGED}, G)|\big)\big)$, where $N_k(\mathsf{CHANGED}, G)$ denotes the k-hop neighborhood of* CHANGED *in G. Otherwise, the algorithm is* unbounded.

Note that Definition 7 only requires the existence of a constant k, and does not impose that k must be small. Therefore, an incremental algorithm remains bounded as long as k is independent of the input size, even if the required neighborhood is relatively large but fixed. For the D-truss decomposition problem, two edges are adjacent if they participate in a common cycle or flow triangle. Hence, $N_k(\cdot)$ is defined on the corresponding triangle-induced edge graph.

Analysis framework. Our analysis is conducted under the *locally persistent* (LP) computation model, which is widely adopted in the study of incremental graph algorithms [2, 12–14, 40]. In this model, each edge maintains local state information and pointers to adjacent edges. No global state is preserved across updates, and all maintenance operations are performed through localized, edge-level accesses.

Under the LP model, D-truss decomposition reduces to maintaining the anchored cycle order as the flow threshold increases. The key question is therefore whether the transformation from O_{k_f} to O_{k_f+1} can be performed by accessing only a bounded neighborhood around the affected edges. The following theorem answers this question in the affirmative.

Theorem 5.1 (Boundedness of Anchored Cycle Order Evolution). *Given a directed graph G and its anchored cycle order O_{k_f} at flow threshold k_f, updating the order to O_{k_f+1} after increasing the flow threshold by one is bounded.*

Proof. Increasing the flow threshold from k_f to $k_f + 1$ removes all edges in the k_f-flow class. Let ΔG denote this set of removed edges. The anchored cycle order update consists of removing ΔG from O_{k_f} and adjusting the positions of the remaining edges whose anchored cycle trussness decreases.

Consider an edge e that is affected during this process. Its anchored cycle trussness may decrease for one of the following two reasons: (1) some anchored cycles supporting e are destroyed due to edge removals, or (2) the number of flow neighbors supporting its current trussness level falls below the requirement imposed by flow threshold $k_f + 1$, violating Property 1(2). If neither condition holds, the anchored cycle trussness of e remains unchanged and no further propagation is triggered. Whenever the anchored cycle trussness of an edge decreases or the edge is removed, it is added to CHANGED. Only in this case do we examine its adjacent edges to check whether they still satisfy the anchored cycle trussness conditions under $k_f + 1$. Edges whose trussness remains valid do not trigger any further neighbor visits.

As a result, the update process visits only edges in CHANGED and their one-hop neighbors. The total maintenance cost is therefore bounded by $O\big(|\mathsf{CHANGED}| + |N_1(\mathsf{CHANGED}, G)|\big)$, which is polynomial in the size of the local neighborhood affected by the update. Under the locally persistent computation model, this establishes that the transformation from O_{k_f} to O_{k_f+1} is bounded. □

5.3 Order-aware Refinement Algorithm

The boundedness of anchored cycle order evolution implies that, when the flow threshold increases, only a small subset of edges can change their relative positions in the order. To exploit this property algorithmically, we develop an *order-aware refinement* strategy that explicitly leverages local order information to restrict the scope of updates.

The key idea is to characterize, for each edge, how much *support* it receives from neighboring edges that are currently ranked higher in the anchored cycle order. Instead of repeatedly recomputing anchored cycle trussness through full local refinement, we maintain two auxiliary counters that allow us to determine whether an edge can preserve its current position in the order.

Auxiliary variables. Fix a flow threshold k_f and an edge e with anchored cycle trussness $\tau_c(e, k_f)$. We define two order-aware support measures for e.

- **Cycle-dominant support.** Let $\mathsf{CS}(e)$ denote the number of cycle triangles containing e such that the anchored cycle trussness of the other two edges in the cycle is no smaller than $\tau_c(e, k_f)$.
- **Flow-dominant support.** Let $\mathsf{FS}(e)$ denote the number of flow triangles containing e in which the anchored cycle trussness of the other two edges is no smaller than $\tau_c(e, k_f)$.

Both $\mathsf{CS}(e)$ and $\mathsf{FS}(e)$ are defined with respect to the current anchored cycle order O_{k_f} and capture the extent to which e is supported by higher-ranked edges in the order. These auxiliary variables enable a more selective refinement process. When the flow threshold increases from k_f to $k_f + 1$, only edges whose supporting structures are affected by edge removals need to be reconsidered.

Lemma 5.2 (Order-aware Stability Condition). *Let e be an edge with anchored cycle trussness $\tau_c(e, k_f)$ at flow threshold k_f. When the flow threshold increases to $k_f + 1$, we have $\tau_c(e, k_f + 1) = \tau_c(e, k_f)$, unless at least one of the following conditions holds: (1) $\mathsf{CS}(e) < \tau_c(e, k_f)$; or (2) $\mathsf{FS}(e) < k_f + 1$.*

Proof. Increasing the flow threshold from k_f to $k_f + 1$ affects an edge e only by removing supporting structures. If e remains supported by at least $\tau_c(e, k_f)$ anchored cycles and satisfies the flow constraint at threshold $k_f + 1$, then its anchored cycle trussness remains valid and unchanged. A decrease can occur only when one of these two conditions is violated, which corresponds exactly to cases (1) or (2). □

As a direct consequence of Lemma 5.2, refinement is invoked only for edges whose order-dominant support becomes insufficient. Edges whose relative positions in the anchored cycle order are guaranteed to remain unchanged do not trigger further propagation, thereby avoiding redundant updates to their neighbors. By explicitly aligning refinement decisions with anchored cycle order information, the proposed order-aware refinement algorithm confines all updates to a small, bounded region of the graph.

Algorithm 4: Order-aware Refinement

Input: Graph $G = (V, E)$ and O_{k_f}
Output: O_{k_f+1}

1 Flag $\leftarrow$ **false**, $active \leftarrow \emptyset$, $H \leftarrow (0, k_f)$-truss of G;
2 **for** $\Phi_{k_f} \neq \emptyset \wedge e \in \Phi_{k_f}$ **do**
3 `UpdateSupports`$(e, k_f, k_f + 1, 0)$;
4 Remove e from H, Flag $\leftarrow$ **true**;
5 **for** $e \in H$ **do**
6 $\tau_c(e, k_f + 1) \leftarrow \tau_c(e, k_f)$;
7 **if** FS$(e) < k_f + 1$ **then** $active[e] \leftarrow$ **true**, Flag $\leftarrow$ **true**;
8 **while** *Flag* **do**
9 *Flag* $\leftarrow$ **false**;
10 **for** $e \in H \wedge active[e] =$ ***true*** **do**
11 $temp \leftarrow min\{C_e(k_f + 1), \mathcal{F}_e(k_f + 1)\}$;
12 $active[e] \leftarrow$ **false**;
13 **if** $temp < \tau_c(e, k_f + 1)$ **then**
14 $\tau_c(e, k_f + 1) \leftarrow temp$, *Flag* $\leftarrow$ **true**;
15 `UpdateSupports`$(e, k_f + 1, k_f + 1, temp)$;
16 **return** O_{k_f+1} ;
17 **Function** `UpdateSupports`(e, k_f, k_f^{cur}, val)**:**
18 **for** *each* $(e_1, e_2) \in \triangle_c(e)$ **do**
19 $m \leftarrow \min\{\tau_c(e, k_f), \tau_c(e_1, k_f), \tau_c(e_2, k_f)\}$;
20 **for** $x \in \{e_1, e_2\}$ **do**
21 **if** $m \geq \tau_c(x, k_f)$ ***and*** $\tau_c(x, k_f) > val$ **then**
22 CS$(x) \leftarrow$ CS$(x) - 1$;
23 **if** CS$(x) < \tau_c(x, k_f)$ **then**
24 $active[x] \leftarrow$ **true**;
25 **for** *each* $(e_1, e_2) \in \triangle_f(e)$ **do**
26 $m \leftarrow \min\{\tau_c(e, k_f), \tau_c(e_1, k_f), \tau_c(e_2, k_f)\}$;
27 **for** $x \in \{e_1, e_2\}$ **do**
28 **if** $m \geq \tau_c(x, k_f)$ ***and*** $\tau_c(x, k_f) > val$ **then**
29 FS$(x) \leftarrow$ FS$(x) - 1$;
30 **if** FS$(x) < k_f^{cur}$ **then**
31 $active[x] \leftarrow$ **true**;

Algorithm 4 presents the pseudocode of the proposed *order-aware refinement* approach. The initial anchored cycle order O_0 is obtained via a peeling-based $(k_c, 0)$-truss decomposition, during which the auxiliary variables CS and FS are computed as byproducts. The overall workflow follows Algorithm 2. To compute the order at flow level $k_f + 1$, the algorithm first removes all edges in the k_f flow class and then refines the anchored cycle trussness of the remaining edges. The key difference lies in a more fine-grained control over the set of edges marked as *active*. Specifically, when the flow threshold increases, the *active* set is divided into two categories. The first consists of edges that violate Lemma 5.2 due to the removal of edges in the k_f flow class (lines 2–4). The second consists of edges that violate Case (2) of Lemma 5.2 solely as a consequence of the increased flow threshold (lines 5–7). The function `UpdateSupports` determines whether the update to an edge e's anchored cycle trussness impacts its neighbors by inspecting the auxiliary variables, and marks affected edges accordingly (lines 17–31). Once an edge is marked as *active*, its anchored cycle trussness is refined through two update operations. The algorithm further examines whether this update affects its neighboring edges, thereby propagating the changes as needed (lines 8–15). It is worth noting that both cycle-based and flow-based updates also modify the corresponding auxiliary variables (line 11).

Time complexity. According to Theorem 5.1, the time complexity of Algorithm 4 is $O\big(|\texttt{CHANGED}| + |N_1(\texttt{CHANGED}, G)|\big)$, where the size of CHANGED is polynomially bounded by the differences between two consecutive anchored cycle orders. When Algorithm 3 incorporates Algorithm 4, the overall time complexity becomes

$$O\left(m^{1.5} + \sum_{i=0}^{k_{f_{\max}}-1} \big(|\texttt{CHANGED}_i| + |N_1(\texttt{CHANGED}_i, G)|\big)\right),$$

where $\texttt{CHANGED}_i$ denotes the set of edges whose anchored cycle orders differ between O_i and O_{i+1}. For D-truss decomposition, adjacency is defined via shared cycle or flow triangles. Under a degeneracy ordering, each edge participates in at most $O(\alpha)$ triangles, where α is the degeneracy of G. Hence, $|N_1(\texttt{CHANGED}_i, G)| = O(\alpha \cdot |\texttt{CHANGED}_i|)$. Since $|\texttt{CHANGED}_i| < m$, we have $O\big(|\texttt{CHANGED}_i| + |N_1(\texttt{CHANGED}i, G)|\big) \subseteq O(m\alpha)$. Therefore, the overall time complexity is bounded by $O\big(m^{1.5} + k_{f_{\max}} \cdot m \cdot \alpha\big)$.

Discussion. Although the current implementation of PACT/PACO is sequential, the proposed framework offers substantial opportunities for parallelization. Triangle enumeration and support initialization can directly leverage existing multicore parallel triangle counting and enumeration frameworks [45]. Similarly, the peeling stage can process removable edges concurrently within the same support level, following existing parallel truss decomposition frameworks [23, 44]. For the refinement phase, each edge update depends only on its local triangle neighbors, enabling synchronized parallel iterative updates while preserving the convergence behavior characterized by Property 1 and Lemma 4.1. Developing efficient parallel implementations of PACT/PACO is an important direction for future work.

6 EXPERIMENTS

We now present the experimental results. Section 6.1 discusses the setup. We report the results in Sections 6.2 and 6.3.

6.1 Setup

Datasets. We conduct our experiments on ten large real-world directed graphs spanning communication, e-commerce, social, knowledge, and web domains. OpenFlights contains ties between two non-US-based airports. Advogato is a social network that lists all user-to-user links. Email-EuAll represents an email communication

network, Amazon is an e-commerce interaction graph, and Pokec, LiveJournal, and Slashdot are social networks. Wikipedia-Link and DBpedia Links are knowledge networks. Enwiki-2013 is a hyperlink graph extracted from the English Wikipedia dump, and Webbase is a large web graph. These datasets are sourced from SNAP[1], LAW[2], and Konect[3]. Table 4 summarizes their statistics, where n denotes the number of vertices, m the number of edges, and $k_{c_{max}}$ and $k_{f_{max}}$ the maximum cycle and flow trussnesses.

Table 4: Directed graphs used in the experiments.

Graphs	Abbr.	Category	n	m	$k_{c_{max}}$	$k_{f_{max}}$
OpenFlights	OF	Infrastructure	2.93K	30.5K	21	63
Advogato	AD	Social	6.54K	51.1K	9	27
Email-EuAll	EM	Communication	0.27M	0.42M	12	39
Slashdot	SD	Social	82.17K	0.94M	33	99
Amazon	AM	Product	0.40M	3.20M	9	27
BerkStan	BS	Web	0.68M	7.60M	161	483
Wikipedia-Link	WL	Hyperlink	0.75M	20.54M	668	2,004
Pokec	PO	Social	1.63M	30.62M	18	54
Live Journal	LJ	Social	4.85M	68.48M	247	750
Enwiki-2013	EW	Text	4.21M	101.35M	29	93

Algorithms. We test the following D-truss decomposition algorithms:

- **`Peeling` [31, 49]:** the state-of-the-art sequential D-truss decomposition algorithm. We use the authors' implementation[4];
- **`Repeel` [29]:** Originally proposed for D-truss retrieval in streaming graphs. We adapt it to static full D-truss decomposition and use the authors' released implementation[5];
- **`PACT`:** Our proposed D-truss decomposition method. The full procedure is presented in Algorithm 3;
- **`PACO`:** Our proposed D-truss decomposition method in Algorithm 3 incorporates order-based refine Algorithm 4.

Experimental settings. In the experiments, we implement all the aforementioned algorithms in C++ and compile them with the g++ 13.3.0 compiler using the -O3 optimization level. The experiments are run on a Linux machine running Ubuntu Linux 20.04.5 LTS. This machine is equipped with dual Intel Xeon(R) Gold 6338 2.0GHz processors (32 cores) and 496GB of RAM. We terminate the algorithm's execution when the running time exceeds 10^8 ms (10^5 s) and mark it as **INF**.

Implementation details. Since triangle enumeration and set intersection operations constitute the main computational cost of the proposed methods, their implementation can significantly affect practical performance. In our implementation, vertices are ordered by their IDs, adjacency lists are maintained in sorted order, and set intersections are performed using the standard two-pointer technique. For cycle triangles, we leverage vertex ordering to avoid redundant enumeration following standard forward-style strategies. For flow triangles, ordering-based deduplication is not directly applicable due to directional constraints, and triangles are enumerated through directed in/out neighbor intersections. These implementation choices are standard in triangle-based graph processing and are orthogonal to the proposed algorithmic framework.

[1]https://snap.stanford.edu/data/index.html
[2]https://law.di.unimi.it/index.php
[3]http://konect.cc/networks/
[4]`Peeling`: https://github.com/TestCodeHouse/DTrussMain/tree/main/ddecomp
[5]`Repeel`: https://github.com/hkbudb/streaming-dtruss/tree/main

6.2 Efficiency evaluation

• **Overall efficiency results.** We evaluate the runtime of all D-truss decomposition algorithms on all datasets, with results reported in Fig. 4. Our `PACO` algorithm consistently achieves the best performance across all datasets, with particularly pronounced advantages on large-scale graphs such as WL, LJ, and EW. Specifically, `PACO` is up to two orders of magnitude faster than the state-of-the-art algorithm `Peeling` on smaller datasets such as EM and SD. Although the streaming-based algorithm `Repeel` improves upon `Peeling`, `PACO` remains substantially faster, especially on large datasets. For example, on WL, `Repeel` requires over 10^5 seconds, whereas `PACO` achieves up to two orders of magnitude speedup. Compared with `PACT`, `PACO` also yields noticeable acceleration, ranging from several-fold to tens-fold across different datasets; in particular, on WL, `PACO` is 44.36× faster than `PACT`. This performance gain primarily stems from the order-aware refinement strategy in `PACO`, which avoids scanning all edges in each iteration and significantly reduces redundant computation.

• **Memory consumption.** We further evaluate the memory usage of all algorithms by recording their peak memory consumption during execution. In addition, Table 5 reports the memory required for loading the graph into memory (`Size`) and the peak memory consumption during triangle enumeration (`TriEnum`). The results show that triangle enumeration itself introduces substantial memory overhead, typically comparable to the raw graph storage size. Compared with `PACT`, `PACO` incurs only slightly higher memory consumption due to the additional auxiliary structures `CS` and `FS`. However, these structures are reused across different k_f values, and only a small number of extra variables are maintained per edge. Therefore, the additional memory overhead remains moderate. From the memory usage of `Peeling` and `Repeel`, we observe that the dominant overhead mainly comes from storing anchored cycle trussness values for different k_f together with the auxiliary variables required during the peeling process.

Table 5: Memory consumption (Bytes).

Dataset	`Size`	`TriEnum`	`Peeling`	`Repeel`	`PACT`	`PACO`
OF	4M	25M	30M	30M	36M	37M
AD	5M	15M	22M	22M	24M	24M
EM	32.4M	90.4M	0.19G	0.19G	0.165G	0.167G
SD	34.4M	0.21G	0.72G	0.72G	0.548G	0.55G
AM	0.13G	0.75G	0.93G	0.93G	1.18G	1.19G
BS	0.27G	5.4G	–	27.96G	19.24G	19.26G
WL	0.64G	72.7G	–	–	226.35G	226.42G
PO	0.99G	7.1G	–	14.67G	13.57G	16.32G
LJ	2.31G	43.25G	–	387.97G	235.34G	235.63G
EW	3.18G	26.9G	–	77.64G	63.17G	63.56G

• **Ablation Study.** To evaluate the contributions of progressive refinement, pruning, and order-aware optimization, we compare four variants: `Repeel`, `PACT w/o pruning`, `PACT`, and `PACO`. Table 6 reports the speedup ratios relative to `Repeel`. The results show that

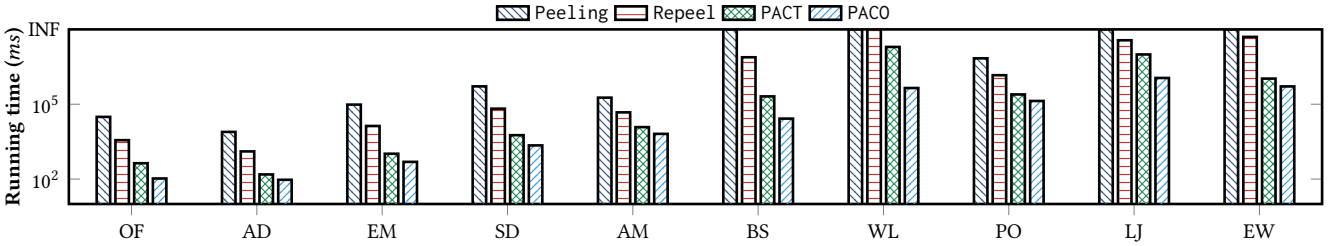


Figure 4: Efficiency of all D-truss decomposition algorithms on all datasets.

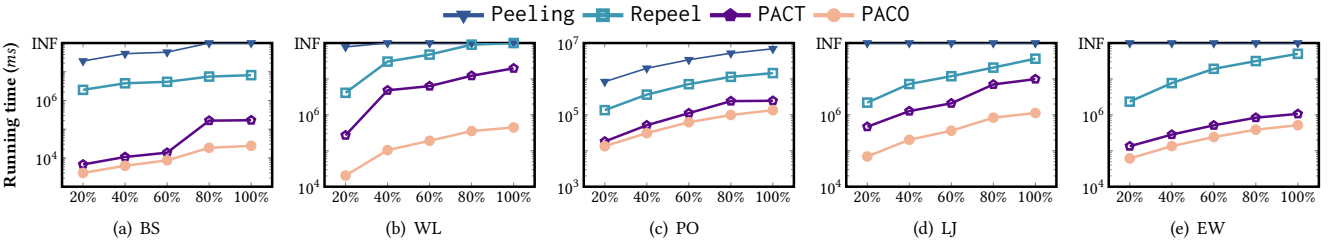


Figure 5: Scalability test.

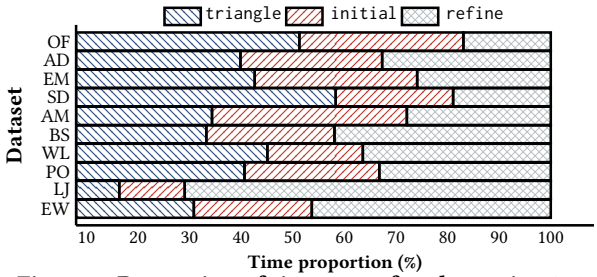


Figure 6: Proportion of time cost of each step in PACO.

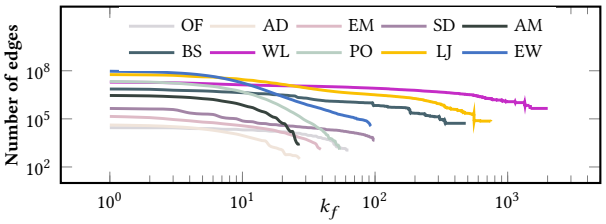


Figure 7: Trends of number of edges of $(0, k_f)$-truss on all datasets.

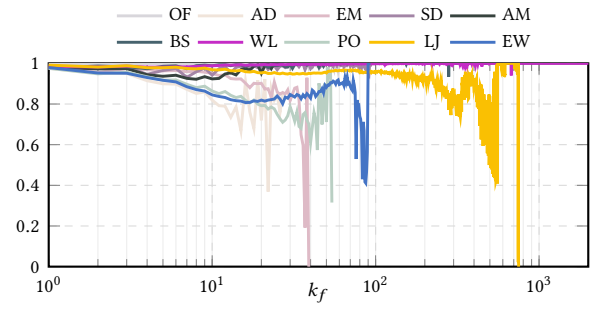


Figure 8: Ratio of edges with unchanged anchored cycle trussness across flow classes.

progressive refinement already provides substantial acceleration over repeated peeling. In particular, PACT w/o pruning achieves up to 47.04× speedup on EW, demonstrating the effectiveness of reusing intermediate decomposition states across flow levels. After

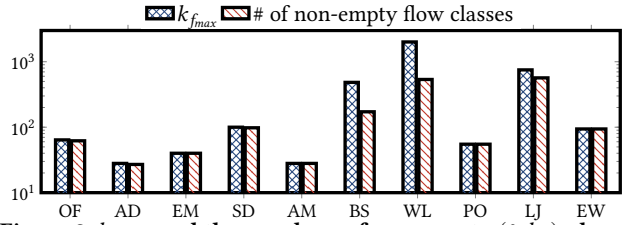


Figure 9: $k_{f_{max}}$ and the numbers of non-empty $(0, k_f)$-classes on all datasets.

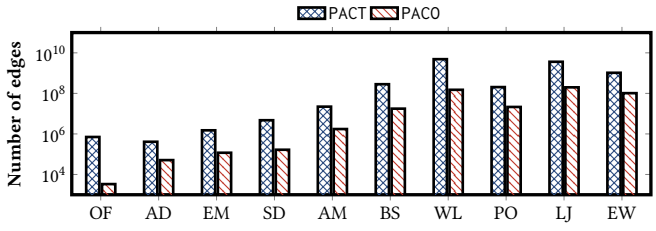


Figure 10: The number of processed edges during refinement.

incorporating the pruning strategy, PACT further improves performance on datasets with many empty or stable flow classes, such as BS and WL. Building upon PACT, the full PACO further achieves significant additional acceleration through the order-aware optimization strategy, reaching up to 287.83× on BS and 224.36× on WL. These results demonstrate that the order-aware optimization substantially reduces unnecessary update propagation and maintenance overhead beyond progressive refinement alone.

• **Scalability test.** For scalability testing, we randomly select 20%, 40%, 60%, 80%, and 100% of edges from each dataset, creating five subgraphs induced by these respective edge percentages. Due to limited space, we present results solely for the six graphs with the largest edge counts, as similar trends prevail across other datasets. As depicted in Figure 5, the time costs for all algorithms increase as the dataset size expands. Notably, among the algorithms, our PACO

Table 6: Speedup ratio to Repeel.

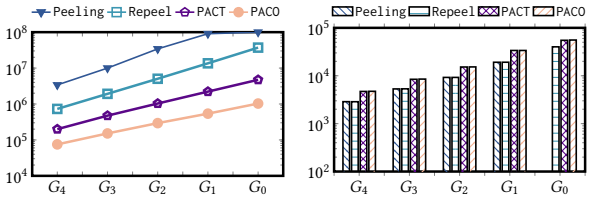
Dataset	Repeel	PACT w/o pruning	PACT	PACO
OF	1×	8.40×	9.39×	34.31×
AD	1×	8.38×	6.69×	13.76×
EM	1×	12.95×	9.56×	27.42×
SD	1×	11.57×	9.19×	29.73×
AM	1×	3.91×	3.18×	7.27×
BS	1×	37.06×	63.54×	287.83×
WL	1×	5.06×	13.37×	224.36×
PO	1×	5.90×	5.69×	10.78×
LJ	1×	3.70×	3.64×	32.50×
EW	1×	47.04×	45.33×	96.84×

demonstrates the least variability in its performance changes, highlighting its superior scalability. Conversely, Peeling and Repeel exhibit relatively efficient performance with smaller datasets. However, as the graph size grows, their runtimes notably increase due to these algorithms' heightened sensitivity to vertex degrees.

• **Time cost of different steps in PACO.** The overall runtime of PACO consists of three phases: (1) computing all triangles in the graph to obtain the *csup* and *fsup* values for each edge (`triangle`); (2) initializing $k_c = 0$ and $k_f = 0$ to determine the flow class and the initial anchored cycle trussness of each edge (`initial`); and (3) iteratively refining the anchored cycle trussness for each k_f until all trussness values are computed (`refine`). Figure 6 presents the time breakdown of these phases across all datasets. Overall, the relative cost of each phase varies markedly across datasets. On smaller datasets, such as OF and AD, triangle computation and anchored cycle trussness initialization dominate the total runtime, since the refinement cost remains limited. In contrast, on large-scale and structurally complex datasets such as WL, the `refine` phase becomes the primary bottleneck, accounting for over 70% of the total runtime. These differences are closely related to the structural properties and degree distributions of the underlying graphs.

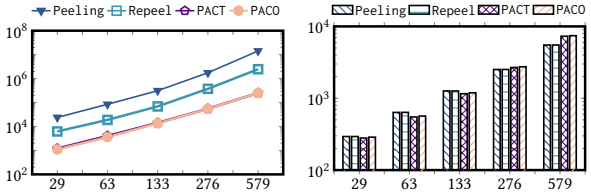
Overall, phase (2) consistently accounts for a relatively small fraction of the total runtime compared to phases (1) and (3), which benefits from efficient peeling strategies such as bin sorting, especially when triangle computation results can be stored and reused. For certain datasets, such as SD, the cost of triangle computation is non-negligible, whereas for others, including LJ and EW, the `refine` phase dominates the runtime. These differences are closely related to the structural properties and degree distributions of the underlying graphs, as further analyzed in the subsequent experiments.

• **Cross-Level Stability.** To evaluate the amount of reusable computation across consecutive flow classes, we measure the proportion of edges whose anchored cycle trussness values remain unchanged between adjacent k_f levels. Figure 8 reports the results on all datasets. Overall, anchored cycle trussness exhibits strong continuity during the progressive refinement process. For most datasets and most k_f values, more than 80% of edges remain unchanged between consecutive flow classes, indicating that only a small fraction of edges require updates. In some cases, the unchanged ratio even reaches 1.0 because the corresponding flow classes are empty. We also observe several exceptions. On the PO dataset, the unchanged ratio decreases in the final refinement stages,



(a) Time cost (ms) (b) Space cost (MB)

Figure 11: Effect of triangle density.



(a) Time cost (ms) (b) Space cost (MB)

Figure 12: Effect of degeneracy.

while on EM and LJ it drops to 0 at the maximum k_f, meaning that all remaining edges are updated simultaneously. These results show that although worst-case propagation may occur in certain high-k_f regions, anchored cycle trussness remains highly stable across adjacent flow classes in most practical cases, which explains the effectiveness of progressive refinement.

• **Robustness Evaluation.** To further evaluate scalability under dense graph structures, we conduct additional experiments on synthetic directed graphs with controlled triangle density and degeneracy. For triangle density, we construct five block-structured directed graphs $\{G_0, \ldots, G_4\}$, each containing 100,000 vertices. The graphs share similar scales but exhibit progressively decreasing triangle densities. In particular, G_0 contains approximately 10^9 directed triangles, and each subsequent graph contains roughly half as many triangles as the previous one. The results in Figure 11 show that the running time of all methods increases nearly linearly with the number of triangles. Nevertheless, PACO consistently achieves the best performance across all datasets. In terms of memory consumption, PACO incurs moderately higher overhead because dense graphs contain fewer empty flow classes and require additional auxiliary structures for order-aware maintenance.

For degeneracy, we further generate a series of directed Erdős–Rényi graphs with 100,000 vertices and progressively increasing edge numbers. The corresponding degeneracy values are $\{29, 63, 133, 276, 579\}$. The results in Figure 12 show that the running time increases smoothly with degeneracy, demonstrating the robustness of the proposed method under increasingly dense graph cores.

• **Anchored k_f vs. achored k_c.** Our implementation fixes k_f and progressively computes anchored cycle trussness values because this decomposition order is consistent with the setting adopted in the existing baseline work [49], enabling a direct and fair comparison. Nevertheless, the proposed framework itself is symmetric and does not intrinsically depend on fixing k_f. The same progressive strategy can also be applied by fixing k_c and iterating over k_f

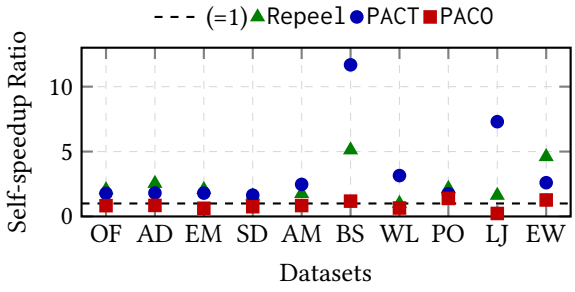


Figure 13: Self-speedup ratio relative to anchored k_f. Points above the dashed line indicate that fixing k_c is faster.

through exchanging the maintained support variables and the corresponding update procedures. To further investigate the effect of dimension selection, we additionally implement the complementary version that fixes k_c instead of k_f. Figure 13 reports the self-speedup ratio of fixing k_c relative to fixing k_f. The results show that the effectiveness of selecting k_c or k_f is graph-dependent. For PACT and Repeel, fixing k_c often performs better on datasets with significantly larger $k_{f_{\max}}$ values. However, for PACO, the performance difference between the two strategies is generally limited. This indicates that the progressive refinement and order-aware optimization substantially reduce the sensitivity to the selected decomposition dimension.

• **Comparing the sizes of the graphs processed.** Figure 9 reports the number of non-empty flow classes and the corresponding $k_{f_{max}}$ for each dataset. In most cases, the number of non-empty flow classes closely matches k_{max}. However, for certain datasets, such as BS and LJ, a noticeable gap is observed between these two values. This phenomenon is mainly attributed to the degree distributions of real-world graphs, which often follow a power-law pattern. As a result, such graphs typically contain a highly dense core that is sharply separated from the surrounding sparse regions. Figure 7 further illustrates the number of edges contained in each $(0, k_f)$-truss across all datasets. It is evident that the graph size processed by PACT decreases substantially as k_f increases. In particular, when k_f approaches $k_{f_{max}}$, the dense region of the graph shrinks significantly, leading to much smaller subgraphs being processed.

• **Comparison of edge accesses during refinement.** In this experiment, we analyze the number of edges accessed during refinement, which reflects the internal execution behavior of the algorithms. As shown in Fig. 10, PACO significantly reduces edge accesses throughout the refinement process, suggesting that its performance gains are primarily attributed to improved per-iteration efficiency. For example, on the OF dataset, PACO processes two orders of magnitude fewer edges than PACT, resulting in a 4× speedup despite the relatively small graph size. On the WL dataset, PACO accesses 32.42× fewer edges than PACT. These results help explain the significant runtime improvements achieved by PACO.

• **Number of iterations.** In the worst case, PACO requires at most δ refinement iterations for each flow threshold k_f, where δ denotes the maximum degree of the input graph, yielding an overall time complexity of $O(k_{f_{\max}} \cdot m \cdot \delta)$. In practice, however, the observed number of refinement iterations is substantially smaller. As reported in Table 7, the total number of iterations required to process all k_f values is well below the theoretical upper bound of $k_{f_{\max}} \cdot \delta$. This behavior can be attributed to the incremental nature of our refinement framework. After initializing the anchored cycle trussness, each refinement step builds directly on the results from the previous flow threshold rather than restarting from the original cycle support. Consequently, the trussness of most edges changes only slightly across successive iterations, which significantly limits the number of effective updates and results in a much smaller iteration count in practice.

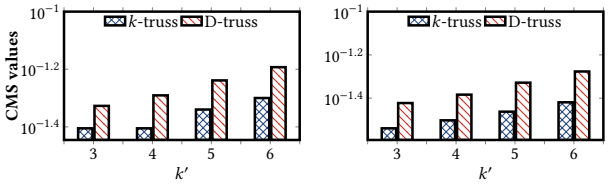


Figure 14: Community comparison of CMS values.

Table 7: The number of iterations required for PACO to compute all anchored cycle trussness with $k_f > 0$.

datasets	OF	AD	EM	SD	AM
# iterations	159	171	300	501	132
δ	237	785	7,631	2,552	2,747
datasets	BS	WL	PO	LJ	EW
# iterations	800	4,557	824	11,413	2,017
δ	84,208	230,446	13,733	20,292	431,795

6.3 Effectiveness evaluations

In this experiment, we apply the D-truss decomposition to construct the index proposed in [31], which is used to support community search queries. We further compare the communities identified by the D-truss model with those obtained using the k-truss-based community model [9].

Specifically, we randomly select a query vertex q and extract a D-truss community containing q by setting the cycle and flow thresholds to the same value, that is, $k_c = k_f$. For comparison, we disregard edge directions and compute the undirected k-truss community containing q with parameter k'. We vary k' over the set $\{3, 4, 5, 6\}$ to obtain communities at different cohesion levels.

To quantitatively evaluate community quality, we adopt the community member similarity (CMS) metric, which measures the structural similarity of vertices within a community in directed graphs [17]. For a community C, CMS is defined as follows:

$$CMS(C)^- = \frac{1}{|C|^2} \sum_{u \in C} \sum_{v \in C} \frac{d^-(u) \cap d^-(v)}{d^-(u) \cup d^-(v)},$$
$$CMS(C)^+ = \frac{1}{|C|^2} \sum_{u \in C} \sum_{v \in C} \frac{d^+(u) \cap d^+(v)}{d^+(u) \cup d^+(v)}. \quad (1)$$

Figure 14 reports the CMS values of communities produced by the two models under different parameter settings. Across all tested values, the D-truss communities consistently achieve higher CMS scores than their k-truss counterparts. This result indicates that, by explicitly incorporating edge directionality, the D-truss model can

identify communities with stronger internal structural similarity and greater cohesiveness.

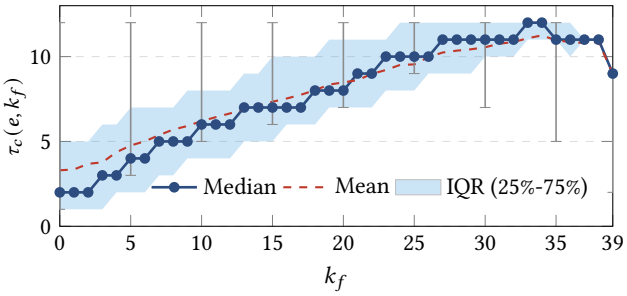


Figure 15: $\tau_c(e, k_f)$ across flow thresholds.

D-truss decomposition provides an effective tool for analyzing structural properties of directed networks. We conduct a case study on the EM dataset to examine how anchored cycle trussness evolves with increasing flow thresholds. In Figure 15, the solid blue curve reports the median $\tau_c(e, k_f)$, the dashed green curve shows the mean, the shaded region corresponds to the interquartile range (IQR, 25th–75th percentiles), and the vertical error bars indicate the full range between the minimum and maximum values. The results show that both the median and mean $\tau_c(e, k_f)$ increase steadily with k_f, indicating that edges surviving higher flow constraints are increasingly embedded in cohesive cycle structures. Meanwhile, the interquartile range and the overall min–max range shrink noticeably, especially for $k_f > 30$, revealing a clear reduction in structural heterogeneity. These trends show that D-truss decomposition captures the progressive concentration of edges into denser and more uniform subgraphs, thereby offering meaningful insights into the network's structural organization.

7 CONCLUSION

In this paper, we propose a progressive framework for efficient D-truss decomposition in directed graphs that organizes computation along increasing flow thresholds. By exploiting the nested structure of D-trusses, we introduced anchored cycle trussness as a compact and monotonic abstraction for tracking structural support under varying flow constraints, and further developed an order-aware perspective that explains why only localized refinement is necessary. Extensive experiments on large real-world graphs demonstrate that our proposed methods significantly outperform existing methods. As future work, the progressive and order-aware principles underlying our solution may be extended to other parameterized cohesive subgraph models and dynamic graph settings, offering a promising direction for scalable graph analysis under evolving constraints.

REFERENCES

[1] Wei Ai, Canhao Xie, Tao Meng, Jayi Du, and Keqin Li. 2024. A D-truss-equivalence Based Index for Community Search over Large Directed Graphs. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[2] Bowen Alpern, Roger Hoover, Barry K Rosen, Peter F Sweeney, and F Kenneth Zadeck. 1990. Incremental evaluation of computational circuits. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms*. 32–42.

[3] Vladimir Batagelj and Matjaz Zaversnik. 2003. An O (m) algorithm for cores decomposition of networks. *arXiv preprint cs/0310049* (2003).

[4] Francesco Bonchi, Francesco Gullo, Andreas Kaltenbrunner, and Yana Volkovich. 2014. Core decomposition of uncertain graphs. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1316–1325.

[5] Lijun Chang and Lu Qin. 2019. Cohesive Subgraph Computation Over Large Sparse Graphs. In *35th IEEE International Conference on Data Engineering, ICDE 2019*. IEEE, 2068–2071.

[6] Lijun Chang, Jeffrey Xu Yu, Lu Qin, Xuemin Lin, Chengfei Liu, and Weifa Liang. 2013. Efficiently computing k-edge connected components via graph decomposition. In *Proceedings of the 2013 ACM SIGMOD international conference on management of data*. 205–216.

[7] Pei-Ling Chen, Chung-Kuang Chou, and Ming-Syan Chen. 2014. Distributed algorithms for k-truss decomposition. In *2014 IEEE International Conference on Big Data (Big Data)*. IEEE, 471–480.

[8] Yankai Chen, Jie Zhang, Yixiang Fang, Xin Cao, and Irwin King. 2021. Efficient community search over large directed graphs: An augmented index-based approach. In *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*. 3544–3550.

[9] Jonathan Cohen. 2008. Trusses: Cohesive subgraphs for social network analysis. *National security agency technical report* 16, 3.1 (2008), 1–29.

[10] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, Hongzhi Chen, and Guoren Wang. 2022. Fast maximal clique enumeration on uncertain graphs: A pivot-based approach. In *Proceedings of the 2022 International Conference on Management of Data*. 2034–2047.

[11] Maximilien Danisch, Oana Balalau, and Mauro Sozio. 2018. Listing k-cliques in sparse real-world graphs. In *Proceedings of the 2018 World Wide Web Conference*. 589–598.

[12] Wenfei Fan, Chunming Hu, and Chao Tian. 2017. Incremental graph computations: Doable and undoable. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 155–169.

[13] Wenfei Fan and Chao Tian. 2022. Incremental graph computations: Doable and undoable. *ACM Transactions on Database Systems (TODS)* 47, 2 (2022), 1–44.

[14] Wenfei Fan, Xin Wang, and Yinghui Wu. 2013. Incremental graph pattern matching. *ACM Transactions on Database Systems (TODS)* 38, 3 (2013), 1–47.

[15] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29, 1 (2020), 353–392.

[16] Yixiang Fang, Wensheng Luo, and Chenhao Ma. 2022. Densest subgraph discovery on large graphs: Applications, challenges, and techniques. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3766–3769.

[17] Yixiang Fang, Zhongran Wang, Reynold Cheng, Hongzhi Wang, and Jiafeng Hu. 2018. Effective and efficient community search over large directed graphs. *IEEE Transactions on Knowledge and Data Engineering* 31, 11 (2018), 2093–2107.

[18] Yixiang Fang, Kaiqiang Yu, Reynold Cheng, Laks VS Lakshmanan, and Xuemin Lin. 2019. Efficient algorithms for densest subgraph discovery. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1719–1732.

[19] Loukas Georgiadis, Evangelos Kipouridis, Charis Papadopoulos, and Nikos Parotsidis. 2023. Faster computation of 3-edge-connected components in digraphs. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 2489–2531.

[20] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 1311–1322.

[21] Shweta Jain and C Seshadhri. 2020. The power of pivoting for exact clique counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 268–276.

[22] Humayun Kabir and Kamesh Madduri. 2017. Parallel k-truss decomposition on multicore systems. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.

[23] Humayun Kabir and Kamesh Madduri. 2017. Parallel k-truss decomposition on multicore systems. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.

[24] Christine Klymko, David Gleich, and Tamara G Kolda. 2014. Using triangles to improve community detection in directed networks. *arXiv preprint arXiv:1404.5874* (2014).

[25] Elizabeth A Leicht and Mark EJ Newman. 2008. Community structure in directed networks. *Physical review letters* 100, 11 (2008), 118703.

[26] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. 2007. Graph evolution: Densification and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)* 1, 1 (2007), 2–es.

[27] Guojing Li, Yuanyuan Zhu, Junchao Ma, Ming Zhong, Tieyun Qian, and Jeffrey Xu Yu. 2025. TDT: Tensor Based Directed Truss Decomposition. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 1828–1840.

[28] Ronghua Li, Sen Gao, Lu Qin, Guoren Wang, Weihua Yang, and Jeffrey Xu Yu. 2020. Ordering Heuristics for k-clique Listing. *Proc. VLDB Endow.* (2020).

[29] Xuankun Liao, Qing Liu, Xin Huang, and Jianliang Xu. 2024. Truss-Based Community Search over Streaming Directed Graphs. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1816–1829.

[30] Xuankun Liao, Qing Liu, Jiaxin Jiang, Xin Huang, Jianliang Xu, and Byron Choi. 2022. Distributed D-core decomposition over large directed graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1546–1558.

[31] Qing Liu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. Truss-based community search over large directed graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2183–2197.

[32] Linyuan Lü, Tao Zhou, Qian-Ming Zhang, and H Eugene Stanley. 2016. The H-index of a network node and its relation to degree and coreness. *Nature communications* 7, 1 (2016), 10168.

[33] Wensheng Luo, Yixiang Fang, Chunxu Lin, and Yingli Zhou. 2024. Efficient Parallel D-Core Decomposition at Scale. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2654–2667.

[34] Wensheng Luo, Kenli Li, Xu Zhou, Yunjun Gao, and Keqin Li. 2022. Maximum Biplex Search over Bipartite Graphs. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 898–910.

[35] Wensheng Luo, Zhuo Tang, Yixiang Fang, Chenhao Ma, and Xu Zhou. 2023. Scalable algorithms for densest subgraph discovery. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 287–300.

[36] Wensheng Luo, Qiaoyuan Yang, Yixiang Fang, and Xu Zhou. 2023. Efficient Core Maintenance in Large Bipartite Graphs. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–26.

[37] Bingqing Lyu, Lu Qin, Xuemin Lin, Ying Zhang, Zhengping Qian, and Jingren Zhou. 2020. Maximum biclique search at billion scale. *Proceedings of the VLDB Endowment* (2020).

[38] Chenhao Ma, Yixiang Fang, Reynold Cheng, Laks VS Lakshmanan, Wenjie Zhang, and Xuemin Lin. 2020. Efficient algorithms for densest subgraph discovery on large directed graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1051–1066.

[39] Yun Peng, Yitong Xu, Huawei Zhao, Zhizheng Zhou, and Huimin Han. 2020. Most similar maximal clique query on large graphs. *Frontiers of Computer Science* 14 (2020), 1–16.

[40] Ganesan Ramalingam and Thomas Reps. 1996. On the computational complexity of dynamic graph problems. *Theoretical Computer Science* 158, 1-2 (1996), 233–277.

[41] Ahmet Erdem Sarıyüce and Ali Pinar. 2016. Fast Hierarchy Construction for Dense Subgraphs. *Proceedings of the VLDB Endowment* 10, 3 (2016).

[42] Ahmet Erdem Sariyuce, C Seshadhri, and Ali Pinar. 2017. Local algorithms for hierarchical dense subgraph discovery. *arXiv preprint arXiv:1704.00386* (2017).

[43] Ahmet Erdem Sariyuce, C Seshadhri, Ali Pinar, and Umit V Catalyurek. 2015. Finding the hierarchy of dense subgraphs using nucleus decompositions. In *Proceedings of the 24th International Conference on World Wide Web*. 927–937.

[44] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and practically efficient parallel nucleus decomposition. *Proceedings of the VLDB Endowment* 15, 3 (2021), 583–596.

[45] Julian Shun and Kanat Tangwongsan. 2015. Multicore triangle computations without tuning. In *2015 IEEE 31st International Conference on Data Engineering*. IEEE, 149–160.

[46] Shaden Smith, Xing Liu, Nesreen K Ahmed, Ancy Sarah Tom, Fabrizio Petrini, and George Karypis. 2017. Truss decomposition on shared-memory parallel systems. In *2017 IEEE high performance extreme computing conference (HPEC)*. IEEE, 1–6.

[47] Zitan Sun, Xin Huang, Jianliang Xu, and Francesco Bonchi. 2021. Efficient probabilistic truss indexing on uncertain graphs. In *Proceedings of the Web Conference 2021*. 354–366.

[48] Taro Takaguchi and Yuichi Yoshida. 2016. Cycle and flow trusses in directed networks. *Royal Society open science* 3, 11 (2016), 160270.

[49] Anxin Tian, Alexander Zhou, Yue Wang, and Lei Chen. 2023. Maximal D-truss search in dynamic directed graphs. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2199–2211.

[50] Anxin Tian, Alexander Zhou, Yue Wang, Xun Jian, Lei Chen, Yan Zhou, and Chen Zhang. 2025. Distributed Truss Decomposition over Large Directed Graphs: A. Tian et al. *The VLDB Journal* 34, 5 (2025), 59.

[51] Jia Wang and James Cheng. 2012. Truss Decomposition in Massive Networks. *Proceedings of the VLDB Endowment* 5, 9 (2012).

[52] Yue Wang, Ruiqi Xu, Xun Jian, Alexander Zhou, and Lei Chen. 2022. Towards distributed bitruss decomposition on bipartite graphs. *Proceedings of the VLDB Endowment* 15, 9 (2022), 1889–1901.

[53] Tianyang Xu, Zhao Lu, and Yuanyuan Zhu. 2022. Efficient triangle-connected truss community search in dynamic graphs. *Proceedings of the VLDB Endowment*

16, 3 (2022), 519–531.

[54] Jianye Yang, Yun Peng, Dian Ouyang, Wenjie Zhang, Xuemin Lin, and Xiang Zhao. 2023. (p, q)-biclique counting and enumeration for large sparse bipartite graphs. *The VLDB Journal* (2023), 1–25.

[55] Yixing Yang, Yixiang Fang, Xuemin Lin, and Wenjie Zhang. 2020. Effective and efficient truss computation over large heterogeneous information networks. In *2020 IEEE 36th international conference on data engineering (ICDE)*. IEEE, 901–912.

[56] Yikai Zhang and Jeffrey Xu Yu. 2019. Unboundedness and efficiency of truss maintenance in evolving graphs. In *Proceedings of the 2019 International Conference on Management of Data*. 1024–1041.

[57] Yingli Zhou, Qingshuo Guo, Yixiang Fang, and Chenhao Ma. 2024. A Counting-based Approach for Efficient k-Clique Densest Subgraph Discovery. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.